

CB3411 – CRYPTOGRAPHY AND CYBER SECURITY LABORATORY

COMPLETE EXAM PREPARATION GUIDE

All Programs in C Language | With Viva Questions | Memory Tricks | Common Mistakes

No.	Experiment	Type
1A	Caesar Cipher	Classical
1B	Playfair Cipher	Classical
1C	Hill Cipher	Classical
2	Rail Fence – Row & Column Transformation	Classical
3	DES (Data Encryption Standard)	Symmetric
4	AES Algorithm	Symmetric
5	RSA Algorithm	Asymmetric
6	Diffie-Hellman Key Exchange	Asymmetric
7	MD5 Hash	Hash
8	SHA-1 Hash	Hash
9	Digital Signature Standard	Signature

EXPERIMENT 1A: IMPLEMENTATION OF CAESAR CIPHER

1. CONCEPT (Simple Language)

- Caesar Cipher shifts each letter by a fixed number called KEY (or shift).
- If key=3: A→D, B→E, C→F ... Z→C (wraps around)
- Encryption: $\text{new_char} = (\text{char} - 'A' + \text{key}) \% 26 + 'A'$
- Decryption: $\text{original} = (\text{char} - 'A' - \text{key} + 26) \% 26 + 'A'$

2. FULL C PROGRAM

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>

void encrypt(char *text, int key) {
    int i;
    for(i = 0; text[i] != '\0'; i++) {
        if(isalpha(text[i])) {
            char base = isupper(text[i]) ? 'A' : 'a';
            text[i] = (text[i] - base + key) % 26 + base;
        }
    }
}

void decrypt(char *text, int key) {
    int i;
    for(i = 0; text[i] != '\0'; i++) {
        if(isalpha(text[i])) {
            char base = isupper(text[i]) ? 'A' : 'a';
            text[i] = (text[i] - base - key + 26) % 26 + base;
        }
    }
}

int main() {
    char text[100];
    int key;
    printf("Enter plaintext: ");
    scanf("%s", text);
    printf("Enter key (shift): ");
    scanf("%d", &key);
    encrypt(text, key);
    printf("Encrypted: %s\n", text);
    decrypt(text, key);
    printf("Decrypted: %s\n", text);
    return 0;
}
```

3. LINE-BY-LINE EXPLANATION

- `char base = isupper(text[i]) ? 'A' : 'a';` → Handles both upper and lowercase
- `(text[i] - base + key) % 26` → Subtracts 'A' to get 0-25 range, adds key, wraps with %26
- `+ base` → Converts back to ASCII character
- `+26` in decrypt → Prevents negative values before mod

4. SAMPLE INPUT & OUTPUT

Input	Output
Plaintext: HELLO Key: 3	Encrypted: KH00R Decrypted: HELLO

5. HOW TO IDENTIFY IN EXAM

- Keywords: shift, rotate, key=3, Caesar
- Logic: single number key, letters shift by fixed amount
- Formula: $(ch - 'A' + key) \% 26 + 'A'$

6. COMMON MISTAKES

- Forgetting +26 in decryption → negative result
- Not using %26 → characters go beyond Z
- Not handling lowercase separately
- Using `gets()` instead of `scanf` (buffer overflow warning)

7. 1-MINUTE REVISION SUMMARY

Caesar = shift cipher. Key = shift amount. Encrypt: add key mod 26. Decrypt: subtract key mod 26. Always add +26 before mod in decrypt. `isalpha()` skips spaces/numbers. `isupper()` handles case.

8. MEMORY TRICK

■ **Memory Trick: 'Caesar ADDED soldiers' → Encrypt = ADD key. 'He SUBTRACTED enemies' → Decrypt = SUBTRACT key. Always +26 to avoid jail (negative numbers)!**

9. MANUAL OUTPUT CALCULATION

- H=7, E=4, L=11, L=11, O=14 (A=0 index)
- Add key=3: 10,7,14,14,17 → K,H,O,O,R
- HELLO + key 3 = KHOOR ✓

10. VIVA QUESTIONS & ANSWERS

Q: What is Caesar Cipher?

A: A substitution cipher where each letter is shifted by a fixed key value.

Q: What is the key in Caesar Cipher?

A: The number of positions to shift. Julius Caesar used key=3.

Q: Why do we use %26?

A: There are 26 alphabets. %26 ensures wrapping: Z+1 = A.

Q: Is Caesar Cipher secure?

A: No. Only 25 possible keys. Brute force attack is easy.

Q: What type of cipher is Caesar?

A: Monoalphabetic substitution cipher.

Q: What is ROT13?

A: Caesar Cipher with key=13. Self-inverse: encrypt twice = original.

EXPERIMENT 1B: IMPLEMENTATION OF PLAYFAIR CIPHER

1. CONCEPT (Simple Language)

- Uses a 5x5 matrix filled with key letters (I and J combined).
- Encrypts 2 letters at a time (called a DIGRAPH/BIGRAM).
- Rules: Same row → shift right | Same col → shift down | Rectangle → swap corners
- If both letters same → insert X between them. If odd length → append X.

2. FULL C PROGRAM

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>

char matrix[5][5];
int used[26] = {0};

void createMatrix(char *key) {
    int i, j, k = 0;
    char temp[100];
    int idx = 0;
    // Add key letters first
    for(i = 0; key[i]; i++) {
        char c = toupper(key[i]);
        if(c == 'J') c = 'I';
        if(!used[c-'A']) {
            temp[idx++] = c;
            used[c-'A'] = 1;
        }
    }
    // Fill remaining alphabet
    for(i = 0; i < 26; i++) {
        if(i == 9) continue; // skip J
        if(!used[i]) {
            temp[idx++] = 'A' + i;
        }
    }
    // Fill 5x5 matrix
    for(i = 0; i < 5; i++)
        for(j = 0; j < 5; j++)
            matrix[i][j] = temp[k++];
}

void findPos(char c, int *row, int *col) {
    int i, j;
    if(c == 'J') c = 'I';
    for(i = 0; i < 5; i++)
        for(j = 0; j < 5; j++)
            if(matrix[i][j] == c) { *row=i; *col=j; return; }
}
```

```

void encrypt(char *text) {
int i, r1,c1,r2,c2;
int len = strlen(text);
for(i = 0; i < len; i += 2) {
findPos(text[i], &r1, &c1);
findPos(text[i+1], &r2, &c2);
if(r1 == r2) { // same row
text[i] = matrix[r1][(c1+1)%5];
text[i+1] = matrix[r2][(c2+1)%5];
} else if(c1 == c2) { // same col
text[i] = matrix[(r1+1)%5][c1];
text[i+1] = matrix[(r2+1)%5][c2];
} else { // rectangle
text[i] = matrix[r1][c2];
text[i+1] = matrix[r2][c1];
}
}
}

void prepareText(char *src, char *dst) {
int i, j = 0;
for(i = 0; src[i]; i++) {
char c = toupper(src[i]);
if(c == 'J') c = 'I';
if(isalpha(c)) dst[j++] = c;
}
dst[j] = '\0';
// Insert X between repeating pairs
char temp[200]; int k = 0;
for(i = 0; i < j; ) {
temp[k++] = dst[i];
if(i+1 < j && dst[i] == dst[i+1]) {
temp[k++] = 'X';
i++;
} else if(i+1 < j) {
temp[k++] = dst[i+1];
i += 2;
} else { i++; }
}
if(k % 2 != 0) temp[k++] = 'X';
temp[k] = '\0';
strcpy(dst, temp);
}

int main() {
char key[50], text[200], prepared[200];
printf("Enter key: "); scanf("%s", key);
printf("Enter plaintext: "); scanf("%s", text);
createMatrix(key);
prepareText(text, prepared);
printf("Prepared text: %s\n", prepared);
encrypt(prepared);
printf("Encrypted: %s\n", prepared);
}

```

```
return 0;
}
```

3. LINE-BY-LINE EXPLANATION

- `createMatrix()`: Fills 5x5 grid with key (no duplicates, I=J, then remaining alphabet)
- `prepareText()`: Prepares digraphs - inserts X between repeats, pads X if odd length
- `findPos()`: Finds row,col of a letter in the 5x5 matrix
- Same row rule: shift right $\rightarrow (\text{col}+1)\%5$
- Same col rule: shift down $\rightarrow (\text{row}+1)\%5$
- Rectangle rule: swap column indices $\rightarrow \text{matrix}[\text{r1}][\text{c2}]$ and $\text{matrix}[\text{r2}][\text{c1}]$

4. SAMPLE INPUT & OUTPUT

- Key: MONARCHY | Plaintext: HELLO
- Matrix starts: M O N A R C H Y B D E F G I K L P Q S T U V W X Z
- Prepared: HE LX (insert X as LL are same pair $\rightarrow$ H E L X)
- Encrypted output will vary based on matrix positions

5. HOW TO IDENTIFY IN EXAM

- Keywords: 5x5 matrix, digraph, I=J, MONARCHY key, bigram
- Logic: pairs of letters, 3 rules (row/col/rectangle)

6. COMMON MISTAKES

- Forgetting I=J causes wrong matrix
- Not handling repeated letters (LL $\rightarrow$ LXL)
- Forgetting to pad X for odd-length text
- Wrong rectangle rule: must swap COLUMNS, not rows

7. 1-MINUTE REVISION SUMMARY

Playfair = 5x5 matrix + digraph. Key fills matrix (I=J, no duplicates). Prepare text: remove non-alpha, insert X between same pairs, pad X if odd. Encrypt: Same row=right, Same col=down, Rectangle=swap corners.

8. MEMORY TRICK

■ **Memory Trick: 'PLAY in PAIRS' $\rightarrow$ always 2 letters. 'Fair ROWS go Right, COLUMNS go Down, RECTANGLE swaps Corners'. I and J are IDENTICAL twins (merge them).**

9. MANUAL OUTPUT CALCULATION

- Write the 5x5 matrix on paper using the key
- Find positions of both letters in each pair
- Apply the correct rule and find the cipher letters

10. VIVA QUESTIONS & ANSWERS

Q: Why is the Playfair matrix 5x5?

A: 26 letters - 1 (J merged with I) = 25 letters. $5 \times 5 = 25$ cells.

Q: Why are I and J combined?

A: To fit 25 letters into $5 \times 5 = 25$ cells. I and J sound similar.

Q: What is a digraph?

A: A pair of two letters encrypted together.

Q: What is inserted between repeated letters?

A: The letter X is inserted as a separator.

Q: What are the 3 encryption rules?

A: 1) Same row: shift right. 2) Same column: shift down. 3) Rectangle: swap column positions.

Q: Is Playfair more secure than Caesar?

A: Yes, because it encrypts pairs making frequency analysis harder.

EXPERIMENT 1C: IMPLEMENTATION OF HILL CIPHER

1. CONCEPT (Simple Language)

- Hill Cipher uses MATRIX MULTIPLICATION to encrypt.
- Key = NxN matrix. Plaintext split into N-letter blocks.
- Encryption: $C = (K \times P) \bmod 26$ (matrix x column vector)
- Decryption: $P = (K_inverse \times C) \bmod 26$
- For 2x2: $C1 = (k11*p1 + k12*p2) \bmod 26$

2. FULL C PROGRAM (2x2 Key Matrix)

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>

void encrypt(int key[2][2], char *text, char *cipher) {
    int i;
    int len = strlen(text);
    for(i = 0; i < len; i += 2) {
        int p1 = toupper(text[i]) - 'A';
        int p2 = toupper(text[i+1]) - 'A';
        cipher[i] = (key[0][0]*p1 + key[0][1]*p2) % 26 + 'A';
        cipher[i+1] = (key[1][0]*p1 + key[1][1]*p2) % 26 + 'A';
    }
    cipher[len] = '\0';
}

int main() {
    int key[2][2];
    char text[100], cipher[100];
    int i, j;
    printf("Enter 2x2 key matrix (4 values):\n");
    for(i = 0; i < 2; i++)
        for(j = 0; j < 2; j++)
            scanf("%d", &key[i][j]);
    printf("Enter plaintext (even length, uppercase): ");
    scanf("%s", text);
    // Pad if odd
    if(strlen(text) % 2 != 0) strcat(text, "X");
    encrypt(key, text, cipher);
    printf("Encrypted: %s\n", cipher);
    return 0;
}
```

3. LINE-BY-LINE EXPLANATION

- $p1 = \text{text}[i] - 'A' \rightarrow$ Convert letter to number (A=0, B=1, ...)
- $\text{key}[0][0]*p1 + \text{key}[0][1]*p2 \rightarrow$ Matrix multiplication for first letter
- $\% 26 \rightarrow$ Keeps result in alphabet range

- + 'A' → Convert number back to letter
- i += 2 → Process 2 letters at a time

4. SAMPLE INPUT & OUTPUT

- Key matrix: $\begin{bmatrix} 3 & 3 \\ 2 & 5 \end{bmatrix}$ Plaintext: HELP
- H=7, E=4: $C1 = (3 \cdot 7 + 3 \cdot 4) \% 26 = (21 + 12) \% 26 = 33 \% 26 = 7 = H$
- $C2 = (2 \cdot 7 + 5 \cdot 4) \% 26 = (14 + 20) \% 26 = 34 \% 26 = 8 = I$
- L=11, P=15: $C1 = (3 \cdot 11 + 3 \cdot 15) \% 26 = (33 + 45) \% 26 = 78 \% 26 = 0 = A$
- Encrypted: HIAP (approx)

5. HOW TO IDENTIFY IN EXAM

- Keywords: matrix, key matrix, matrix multiplication, mod 26
- Logic: 2 or 3 letters at a time, uses multiplication

6. COMMON MISTAKES

- Not padding odd-length text with X
- Forgetting % 26 after multiplication
- Key matrix must be invertible mod 26 (det must not be 0 or share factor with 26)
- Confusing row-major vs column vector multiplication

7. 1-MINUTE REVISION SUMMARY

Hill Cipher = Matrix Multiplication mod 26. Key is NxN matrix. Split plaintext into N-length blocks. Multiply key matrix by plaintext vector mod 26. Convert numbers back to letters. Decrypt using matrix inverse mod 26.

8. MEMORY TRICK

■ **Memory Trick: 'HILL climbers carry MATRICES' → Hill uses matrix. 'Multiply, Mod, Map' → the 3 steps. Key matrix looks like a HILL from the side!**

9. MANUAL OUTPUT CALCULATION

- Convert letters to numbers: A=0 to Z=25
- Multiply key matrix x plaintext vector
- Apply mod 26 to each result
- Convert back to letters

10. VIVA QUESTIONS & ANSWERS

Q: What is Hill Cipher?

A: A polygraphic substitution cipher using matrix multiplication modulo 26.

Q: What size matrix is used?

A: Any NxN matrix. Most commonly 2x2 or 3x3 for practical examples.

Q: What condition must the key matrix satisfy?

A: It must be invertible modulo 26. Its determinant must be coprime to 26.

Q: What is the decryption formula?

A: $P = K_{\text{inverse}} * C \bmod 26$, where K_{inverse} is the modular inverse of key matrix.

Q: Why is Hill Cipher stronger than Caesar?

A: It encrypts multiple letters at once making frequency analysis much harder.

EXPERIMENT 2: RAIL FENCE – ROW & COLUMN TRANSFORMATION

1. CONCEPT (Simple Language)

- Rail Fence: Write plaintext in ZIG-ZAG pattern across N rails, then read row by row.
- Row Transposition: Write text row by row, read column by column (or by key order).
- Column Transposition: Rearrange columns based on alphabetical order of key.

2. FULL C PROGRAM – RAIL FENCE

```
#include<stdio.h>
#include<string.h>

void railFenceEncrypt(char *text, int rails) {
    int len = strlen(text);
    char fence[rails][len];
    int i, j, rail = 0, dir = 1;
    // Initialize
    for(i = 0; i < rails; i++)
        for(j = 0; j < len; j++)
            fence[i][j] = '\0';
    // Fill zigzag
    for(i = 0; i < len; i++) {
        fence[rail][i] = text[i];
        if(rail == 0) dir = 1;
        else if(rail == rails-1) dir = -1;
        rail += dir;
    }
    // Read row by row
    printf("Encrypted: ");
    for(i = 0; i < rails; i++)
        for(j = 0; j < len; j++)
            if(fence[i][j] != '\0') printf("%c", fence[i][j]);
    printf("\n");
}

int main() {
    char text[100];
    int rails;
    printf("Enter plaintext: "); scanf("%s", text);
    printf("Enter number of rails: "); scanf("%d", &rails);
    railFenceEncrypt(text, rails);
    return 0;
}
```

3. LINE-BY-LINE EXPLANATION

- fence[rails][len] → 2D array, each row is a rail
- dir = 1 or -1 → Direction of movement (down or up)

- if(rail==0) dir=1 → Hit top rail, go down
- if(rail==rails-1) dir=-1 → Hit bottom rail, go up
- Read fence row by row to get ciphertext

4. SAMPLE INPUT & OUTPUT

- Input: HELLOWORLD, Rails: 3
- Rail 0: H . . . O . . . L .
- Rail 1: . E . L . W . R . D
- Rail 2: . . L . . . O . . .
- Output: HOLELWRDLO

5. HOW TO IDENTIFY IN EXAM

- Keywords: rail fence, zigzag, rails, transposition
- Logic: text is rearranged (NOT substituted), positional change

6. COMMON MISTAKES

- Not initializing fence array → garbage characters
- Wrong direction change: dir must flip at BOTH boundaries (0 and rails-1)
- Forgetting that Rail Fence is TRANSPOSITION not substitution

7. 1-MINUTE REVISION SUMMARY

Rail Fence = zigzag writing. Write text diagonally across N rails. Read each rail left to right. Direction bounces between top and bottom. Use dir=+1/-1 to track direction. Row/Column transposition = rearrange using key column order.

8. MEMORY TRICK

■ **Memory Trick: 'A train on RAILS goes UP and DOWN (zigzag)' → Rail Fence = zigzag. 'Read the FENCE post by post (rail by rail)' → read row by row for ciphertext.**

10. VIVA QUESTIONS & ANSWERS

Q: What type of cipher is Rail Fence?

A: Transposition cipher. Letters are rearranged, not substituted.

Q: What is the difference between substitution and transposition?

A: Substitution changes the letter. Transposition changes the position of letters.

Q: How many rails are commonly used?

A: 2 or 3 rails in standard examples. More rails = more security.

Q: What is row-column transposition?

A: Write text row-by-row in a grid, then read column by column based on key order.

EXPERIMENT 3: IMPLEMENTATION OF DES (Data Encryption Standard)

1. CONCEPT (Simple Language)

- DES = block cipher. Encrypts 64-bit blocks using a 56-bit key (64-bit with parity).
- Uses 16 rounds of Feistel structure.
- Each round: Expansion, XOR with subkey, S-box substitution, Permutation.
- In C lab: We implement a SIMPLIFIED DES to show the concept.

2. FULL C PROGRAM (Simplified DES using XOR)

```
#include<stdio.h>
#include<string.h>

// Simplified DES demonstration using XOR rounds
void generateSubkeys(unsigned char key, unsigned char subkeys[], int rounds) {
    int i;
    subkeys[0] = key;
    for(i = 1; i < rounds; i++) {
        subkeys[i] = (subkeys[i-1] << 1) | (subkeys[i-1] >> 7);
    }
}

unsigned char feistelRound(unsigned char half, unsigned char subkey) {
    return half ^ subkey;
}

void desEncrypt(unsigned char *left, unsigned char *right, unsigned char subkeys[], int rounds) {
    int i;
    unsigned char temp;
    for(i = 0; i < rounds; i++) {
        temp = *right;
        *right = *left ^ feistelRound(*right, subkeys[i]);
        *left = temp;
    }
}

void desDecrypt(unsigned char *left, unsigned char *right, unsigned char subkeys[], int rounds) {
    int i;
    unsigned char temp;
    for(i = rounds-1; i >= 0; i--) {
        temp = *left;
        *left = *right ^ feistelRound(*left, subkeys[i]);
        *right = temp;
    }
}

int main() {
    unsigned char key, left, right;
```

```

unsigned char subkeys[16];
int rounds = 4;
printf("Enter key (0-255): "); scanf("%hhu", &key);
printf("Enter left half (0-255): "); scanf("%hhu", &left);
printf("Enter right half (0-255): "); scanf("%hhu", &right);
generateSubkeys(key, subkeys, rounds);
printf("\nOriginal: L=%d R=%d\n", left, right);
desEncrypt(&left, &right, subkeys, rounds);
printf("Encrypted: L=%d R=%d\n", left, right);
desDecrypt(&left, &right, subkeys, rounds);
printf("Decrypted: L=%d R=%d\n", left, right);
return 0;
}

```

3. LINE-BY-LINE EXPLANATION

- generateSubkeys(): Creates round keys by rotating bits left each round
- feistelRound(): F-function using XOR with subkey
- desEncrypt(): 4 Feistel rounds: temp=R, R=L XOR F(R,K), L=temp
- desDecrypt(): Same structure but subkeys applied in REVERSE order
- (subkeys[i-1]<<1)|(subkeys[i-1]>>7) → Left circular rotation by 1 bit

4. SAMPLE INPUT & OUTPUT

- Key: 170 Left: 90 Right: 45
- Encrypted: L and R values change after 4 rounds
- Decrypted: Returns to L=90, R=45

5. HOW TO IDENTIFY IN EXAM

- Keywords: DES, Feistel, 16 rounds, 56-bit key, 64-bit block, subkeys
- Logic: Split into Left/Right halves, swap each round

6. COMMON MISTAKES

- Applying subkeys in wrong order during decryption (must be REVERSED)
- Forgetting the Feistel swap: temp = R first, THEN update R, THEN L = temp
- Not generating separate subkeys for each round

7. 1-MINUTE REVISION SUMMARY

DES = Feistel network, 16 rounds, 64-bit blocks, 56-bit key. Each round: split L and R, R = L XOR F(R, subkey), L = old R. Decrypt = same process with reversed subkeys. Feistel magic: decryption is encryption reversed.

8. MEMORY TRICK

■ **Memory Trick: 'DES SPLITS and SWAPS like a juggler (Feistel)'. 'LEFT becomes RIGHT, RIGHT becomes LEFT XOR Key'. Decrypt? Just REVERSE the keys order!**

10. VIVA QUESTIONS & ANSWERS

Q: What is DES?

A: Data Encryption Standard. A symmetric block cipher using 56-bit key and 64-bit blocks.

Q: How many rounds does DES have?

A: 16 rounds of Feistel structure.

Q: What is Feistel structure?

A: Split block into L and R halves. Each round: $\text{new_R} = \text{L XOR } F(\text{R}, \text{subkey})$, $\text{new_L} = \text{old R}$.

Q: Why is DES considered insecure today?

A: 56-bit key is too short. Can be brute-forced in hours with modern computers.

Q: What replaced DES?

A: 3DES (Triple DES) and later AES (Advanced Encryption Standard).

Q: What is the effective key length of DES?

A: 56 bits (8 bits are parity bits in the 64-bit key).

EXPERIMENT 4: IMPLEMENTATION OF AES ALGORITHM

1. CONCEPT (Simple Language)

- AES = Advanced Encryption Standard. Block cipher with 128-bit block, 128/192/256-bit key.
- 4 main operations per round: SubBytes, ShiftRows, MixColumns, AddRoundKey.
- 128-bit key = 10 rounds. 192-bit = 12. 256-bit = 14.
- In lab: We simulate AES steps to demonstrate the concept.

2. FULL C PROGRAM (AES Simplified Simulation)

```
#include<stdio.h>
#include<string.h>

// Simplified AES demonstration
// S-Box (partial, for demonstration)
unsigned char sbox[16] = {
0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5,
0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76
};

void addRoundKey(unsigned char state[], unsigned char key[], int len) {
int i;
for(i = 0; i < len; i++)
state[i] ^= key[i];
}

void subBytes(unsigned char state[], int len) {
int i;
for(i = 0; i < len; i++)
state[i] = sbox[state[i] % 16]; // simplified
}

void shiftRows(unsigned char state[4][4]) {
unsigned char temp;
// Row 1: shift left by 1
temp = state[1][0];
state[1][0] = state[1][1];
state[1][1] = state[1][2];
state[1][2] = state[1][3];
state[1][3] = temp;
// Row 2: shift left by 2
temp = state[2][0]; state[2][0] = state[2][2]; state[2][2] = temp;
temp = state[2][1]; state[2][1] = state[2][3]; state[2][3] = temp;
}

int main() {
unsigned char state[16] = "HELLO AES WORLD!";
unsigned char key[16] = "MYSECRETKEY12345";
int i;
printf("Plaintext: ");
for(i=0;i<16;i++) printf("%c",state[i]);
printf("\n");
```

```
// Round 1 demonstration
addRoundKey(state, key, 16);
printf("After AddRoundKey (hex): ");
for(i=0;i<16;i++) printf("%02x ",state[i]);
printf("\n");
subBytes(state, 16);
printf("After SubBytes (hex): ");
for(i=0;i<16;i++) printf("%02x ",state[i]);
printf("\n");
return 0;
}
```

3. LINE-BY-LINE EXPLANATION

- state[16] = 16 bytes = 128 bits = one AES block
- addRoundKey(): XOR each byte of state with corresponding key byte
- subBytes(): Replace each byte using S-Box lookup table (non-linear)
- shiftRows(): Row 0 no shift, Row 1 shift left 1, Row 2 shift left 2, Row 3 shift left 3
- MixColumns(): Matrix multiplication in $GF(2^8)$ - complex, often omitted in lab

4. SAMPLE INPUT & OUTPUT

- Plaintext: "HELLO AES WORLD!"
- Key: "MYSECRETKEY12345"
- AddRoundKey: XOR of ASCII values printed as hex
- SubBytes: Each byte replaced using S-Box

5. HOW TO IDENTIFY IN EXAM

- Keywords: AES, SubBytes, ShiftRows, MixColumns, AddRoundKey, S-Box
- Logic: 4 step transformation per round, works on 4x4 byte state matrix

6. COMMON MISTAKES

- Confusing AES with DES (AES has no Feistel; uses substitution-permutation network)
- Wrong ShiftRows: Row 0=no shift, Row 1=1, Row 2=2, Row 3=3
- S-Box lookup must use exact values, not approximations

7. 1-MINUTE REVISION SUMMARY

AES = Substitution-Permutation Network. 128-bit block, 10/12/14 rounds. Each round:
 1)SubBytes(S-Box lookup) 2)ShiftRows(rotate rows) 3)MixColumns(matrix multiply)
 4)AddRoundKey(XOR with subkey). Last round skips MixColumns.

8. MEMORY TRICK

■ **Memory Trick: 'Students Should Mix And Repeat' → SubBytes, ShiftRows, MixColumns, AddRoundKey, Repeat. AES is a 4-step dance repeated 10 times!**

10. VIVA QUESTIONS & ANSWERS

Q: What is AES?

A: Advanced Encryption Standard. A symmetric block cipher adopted by NIST in 2001.

Q: What are the 4 operations in each AES round?

A: SubBytes, ShiftRows, MixColumns, AddRoundKey.

Q: How many rounds does AES-128 use?

A: 10 rounds. AES-192 uses 12. AES-256 uses 14.

Q: What is the S-Box in AES?

A: A 16x16 lookup table used in SubBytes for non-linear substitution.

Q: What is the state matrix in AES?

A: A 4x4 matrix of bytes (16 bytes = 128 bits) representing the current block.

Q: What is AddRoundKey?

A: XOR of current state with the round key derived from the original key.

EXPERIMENT 5: IMPLEMENTATION OF RSA ALGORITHM

1. CONCEPT (Simple Language)

- RSA = asymmetric cipher. Uses PUBLIC key to encrypt, PRIVATE key to decrypt.
- Steps: 1) Choose primes p,q 2) $n=p*q$ 3) $\phi=(p-1)*(q-1)$
- 4) Choose e: $\gcd(e, \phi)=1$ 5) Find d: $e*d \bmod \phi = 1$
- Public Key: (e, n) Private Key: (d, n)
- Encrypt: $C = M^e \bmod n$ Decrypt: $M = C^d \bmod n$

2. FULL C PROGRAM

```
#include<stdio.h>
#include<math.h>

long long gcd(long long a, long long b) {
while(b != 0) {
long long temp = b;
b = a % b;
a = temp;
}
return a;
}

long long power(long long base, long long exp, long long mod) {
long long result = 1;
base = base % mod;
while(exp > 0) {
if(exp % 2 == 1)
result = (result * base) % mod;
exp = exp / 2;
base = (base * base) % mod;
}
return result;
}

int main() {
long long p, q, n, phi, e, d, msg, encrypted, decrypted;
printf("Enter prime p: "); scanf("%lld", &p);
printf("Enter prime q: "); scanf("%lld", &q);
n = p * q;
phi = (p-1) * (q-1);
printf("n = %lld, phi = %lld\n", n, phi);
// Choose e
for(e = 2; e < phi; e++)
if(gcd(e, phi) == 1) break;
printf("Public key e = %lld\n", e);
// Find d using extended Euclidean (simplified brute force)
for(d = 1; d < phi; d++)
if((e * d) % phi == 1) break;
printf("Private key d = %lld\n", d);
```

```
printf("Enter message (number < n): "); scanf("%lld", &msg);
encrypted = power(msg, e, n);
decrypted = power(encrypted, d, n);
printf("Encrypted: %lld\n", encrypted);
printf("Decrypted: %lld\n", decrypted);
return 0;
}
```

3. LINE-BY-LINE EXPLANATION

- $n = p \cdot q \rightarrow$ modulus used in both keys
- $\phi = (p-1) \cdot (q-1) \rightarrow$ Euler's totient function
- $\gcd(e, \phi) = 1 \rightarrow e$ must be coprime to ϕ (no common factors)
- $(e \cdot d) \% \phi = 1 \rightarrow d$ is modular inverse of e (brute force method)
- $\text{power}(\text{base}, \text{exp}, \text{mod}) \rightarrow$ Fast modular exponentiation (repeated squaring)
- $C = M^e \bmod n \rightarrow$ Encryption | $M = C^d \bmod n \rightarrow$ Decryption

4. SAMPLE INPUT & OUTPUT

- $p=7, q=11 \rightarrow n=77, \phi=60$
- $e=13$ ($\gcd(13,60)=1$), $d=37$ ($13 \cdot 37=481, 481 \% 60=1$)
- Message=9: Encrypted = $9^{13} \bmod 77 = 48$
- Decrypted = $48^{37} \bmod 77 = 9 \checkmark$

5. HOW TO IDENTIFY IN EXAM

- Keywords: RSA, prime, p , q , n , ϕ , public key, private key, modular exponentiation
- Formula: $C = M^e \bmod n$, $M = C^d \bmod n$

6. COMMON MISTAKES

- Using int instead of long long (overflow with large powers)
- Message M must be less than n
- p and q must be PRIME (not just any numbers)
- Not using modular exponentiation $\rightarrow$ overflow crash

7. 1-MINUTE REVISION SUMMARY

RSA: Pick primes p, q . $n=p \cdot q$. $\phi=(p-1)(q-1)$. Find e : $\gcd(e, \phi)=1$. Find d : $e \cdot d \bmod \phi=1$. Public key= (e, n) , Private key= (d, n) . Encrypt: $C=M^e \bmod n$. Decrypt: $M=C^d \bmod n$. Use long long and modular exponentiation.

8. MEMORY TRICK

■ **Memory Trick: 'Real Security Always needs two PRIMES (p, q)'. 'e ENCRYPTS, d DECRYPTS'. n =Needed by both. $\phi=\Phi$ helps find keys. Remember: e and d are MODULAR INVERSES of each other!**

9. MANUAL OUTPUT CALCULATION

- $p=3, q=5 \rightarrow n=15, \phi=8$
- $e=3$ ($\gcd(3,8)=1$), $d=3$ ($3*3=9, 9\%8=1$)
- $M=7$: $C=7^3 \bmod 15 = 343 \bmod 15 = 13$
- Check: $13^3 \bmod 15 = 2197 \bmod 15 = 7 \checkmark$

10. VIVA QUESTIONS & ANSWERS

Q: What is RSA?

A: Rivest-Shamir-Adleman. An asymmetric encryption algorithm using public-private key pairs.

Q: Why are two primes chosen?

A: Their product n is easy to compute but hard to factor back. Security relies on factoring difficulty.

Q: What is Euler's totient $\phi(n)$?

A: $\phi(n) = (p-1)(q-1)$. Count of integers less than n that are coprime to n .

Q: What is the relationship between e and d ?

A: They are modular multiplicative inverses: $e*d \equiv 1 \pmod{\phi}$.

Q: What makes RSA secure?

A: Factoring large numbers ($n = p*q$) into p and q is computationally infeasible.

Q: Is RSA faster or slower than AES?

A: RSA is much slower. Used only to exchange keys; AES is used for actual data.

EXPERIMENT 6: DIFFIE-HELLMAN KEY EXCHANGE ALGORITHM

1. CONCEPT (Simple Language)

- DH allows two parties to create a SHARED SECRET over a public channel WITHOUT sending the key.
- Public values: prime p and generator g (both known to everyone)
- Alice: private $a \rightarrow$ public $A = g^a \bmod p$
- Bob: private $b \rightarrow$ public $B = g^b \bmod p$
- Shared secret: Alice computes $B^a \bmod p$, Bob computes $A^b \bmod p \rightarrow$ SAME result!

2. FULL C PROGRAM

```
#include<stdio.h>
#include<math.h>

long long power(long long base, long long exp, long long mod) {
    long long result = 1;
    base = base % mod;
    while(exp > 0) {
        if(exp % 2 == 1)
            result = (result * base) % mod;
        exp /= 2;
        base = (base * base) % mod;
    }
    return result;
}

int main() {
    long long p, g, a, b;
    long long A, B, secretAlice, secretBob;
    printf("Enter prime number p: "); scanf("%lld", &p);
    printf("Enter generator g (primitive root of p): "); scanf("%lld", &g);
    printf("Enter Alice's private key a: "); scanf("%lld", &a);
    printf("Enter Bob's private key b: "); scanf("%lld", &b);
    // Compute public keys
    A = power(g, a, p); // Alice's public key
    B = power(g, b, p); // Bob's public key
    printf("Alice's public key A = %lld\n", A);
    printf("Bob's public key B = %lld\n", B);
    // Compute shared secrets
    secretAlice = power(B, a, p); // Alice computes B^a mod p
    secretBob = power(A, b, p); // Bob computes A^b mod p
    printf("Alice's shared secret = %lld\n", secretAlice);
    printf("Bob's shared secret = %lld\n", secretBob);
    if(secretAlice == secretBob)
        printf("Key Exchange Successful! Shared key = %lld\n", secretAlice);
    else
        printf("Key Exchange Failed!\n");
    return 0;
}
```

}

3. LINE-BY-LINE EXPLANATION

- p = large prime (public), g = generator/primitive root of p (public)
- a, b = private keys (kept secret by Alice and Bob respectively)
- $A = g^a \bmod p \rightarrow$ Alice's public key (sent to Bob)
- $B = g^b \bmod p \rightarrow$ Bob's public key (sent to Alice)
- $\text{secretAlice} = B^a \bmod p = (g^b)^a \bmod p = g^{(ab)} \bmod p$
- $\text{secretBob} = A^b \bmod p = (g^a)^b \bmod p = g^{(ab)} \bmod p$
- Both compute the SAME value $g^{(ab)} \bmod p \rightarrow$ shared secret!

4. SAMPLE INPUT & OUTPUT

- $p=23, g=5, a=6$ (Alice's private), $b=15$ (Bob's private)
- $A = 5^6 \bmod 23 = 15625 \bmod 23 = 8$
- $B = 5^{15} \bmod 23 = 19$
- Alice's secret = $19^6 \bmod 23 = 2$
- Bob's secret = $8^{15} \bmod 23 = 2 \checkmark$ (Both get 2!)

5. HOW TO IDENTIFY IN EXAM

- Keywords: Diffie-Hellman, key exchange, primitive root, generator g , prime p
- Logic: No actual message encrypted; just establishing a shared secret key

6. COMMON MISTAKES

- Confusing public and private keys
- Not using modular exponentiation $\rightarrow$ overflow
- Using small primes $\rightarrow$ insecure but OK for lab demos
- Thinking DH encrypts data – it only EXCHANGES KEYS

7. 1-MINUTE REVISION SUMMARY

DH = Key Exchange only (not encryption). Public: p (prime), g (generator). Alice: private a , public $A=g^a \bmod p$. Bob: private b , public $B=g^b \bmod p$. Exchange A and B publicly. Alice: $B^a \bmod p$. Bob: $A^b \bmod p$. Both = $g^{(ab)} \bmod p$ = shared secret!

8. MEMORY TRICK

■ **Memory Trick: 'DH is like mixing paint: Alice+Bob each have secret color. Mix with common color publicly. End result = same secret paint = shared key. Enemy sees mixed paint but can't unmix it!'**

10. VIVA QUESTIONS & ANSWERS

Q: What does Diffie-Hellman do?

A: It allows two parties to establish a shared secret over an insecure channel without prior secret sharing.

Q: What is a primitive root/generator?

A: A number g where $g^1, g^2, \dots, g^{(p-1)} \bmod p$ produces all values from 1 to $p-1$.

Q: Does Diffie-Hellman encrypt data?

A: No. It only establishes a shared key. Actual encryption uses another algorithm like AES.

Q: What attack does DH face?

A: Man-in-the-Middle (MITM) attack if public keys are not authenticated.

Q: Why is DH secure?

A: Security relies on the Discrete Logarithm Problem: given $g^a \bmod p$, finding a is computationally hard.

Q: What is the shared secret formula?

A: $g^{(ab)} \bmod p = B^a \bmod p = A^b \bmod p$.

EXPERIMENT 7: IMPLEMENTATION OF MD5 HASH

1. CONCEPT (Simple Language)

- MD5 = Message Digest 5. Takes ANY input → produces fixed 128-bit (32 hex char) hash.
- One-way: cannot reverse the hash to get original message.
- Even 1 character change → completely different hash (avalanche effect).
- Used for: password storage, file integrity check.

2. FULL C PROGRAM (MD5 Simplified / Using OpenSSL)

```
// METHOD 1: Using OpenSSL library (compile with -lssl -lcrypto)
#include<stdio.h>
#include<string.h>
#include<openssl/md5.h>

int main() {
    char text[256];
    unsigned char digest[MD5_DIGEST_LENGTH];
    int i;
    printf("Enter message: ");
    fgets(text, sizeof(text), stdin);
    text[strcspn(text, "\n")] = 0; // remove newline
    MD5((unsigned char*)text, strlen(text), digest);
    printf("MD5 Hash: ");
    for(i = 0; i < MD5_DIGEST_LENGTH; i++)
        printf("%02x", digest[i]);
    printf("\n");
    return 0;
}

// Compile: gcc md5.c -o md5 -lssl -lcrypto

// ---- OR: Simplified Manual MD5 (no library needed) ----
#include<stdio.h>
#include<stdint.h>
#include<string.h>

// Simple hash demonstration (NOT actual MD5)
void simpleHash(char *msg, unsigned char hash[16]) {
    int i;
    unsigned int state[4] = {0x67452301, 0xEFCDAB89, 0x98BADCFE, 0x10325476};
    unsigned int sum = 0;
    for(i = 0; msg[i]; i++)
        sum += (unsigned char)msg[i] * (i+1);
    state[0] ^= sum;
    state[1] ^= (sum << 5) | (sum >> 27);
    state[2] ^= (sum >> 3);
    state[3] ^= sum * 0x9e3779b9;
    memcpy(hash, state, 16);
}

int main() {
```

```

char msg[256];
unsigned char hash[16];
int i;
printf("Enter message: "); scanf("%s", msg);
simpleHash(msg, hash);
printf("Hash (16 bytes hex): ");
for(i = 0; i < 16; i++) printf("%02x", hash[i]);
printf("\n");
return 0;
}

```

3. LINE-BY-LINE EXPLANATION

- MD5_DIGEST_LENGTH = 16 (bytes) = 128 bits
- MD5() function takes: input string, length, output buffer
- printf('%02x') → prints each byte as 2 hex digits (00 to ff)
- Initial state values {0x67452301...} → MD5 magic initialization constants
- strcspn(text, newline) → removes trailing newline from fgets

4. SAMPLE INPUT & OUTPUT

- Input: "hello"
- MD5 Hash: 5d41402abc4b2a76b9719d911017c592
- Input: "Hello" (capital H)
- MD5 Hash: 8b1a9953c4611296a827abf8c47804d7 (completely different!)

5. HOW TO IDENTIFY IN EXAM

- Keywords: hash, digest, MD5, 128-bit, one-way, message digest
- Output: always 32 hex characters regardless of input size

6. COMMON MISTAKES

- Forgetting to link OpenSSL: must add -lssl -lcrypto in gcc command
- Using scanf instead of fgets for messages with spaces
- Printing hash as %d instead of %02x (wrong format)

7. 1-MINUTE REVISION SUMMARY

MD5 produces 128-bit (16 byte = 32 hex char) hash. One-way function. Use OpenSSL: MD5(input, len, output). Print with %02x. Compile with -lssl -lcrypto. Same input = same hash. Tiny change = completely different hash (avalanche effect).

8. MEMORY TRICK

■ **Memory Trick: 'MD5 = Message Digester that makes 32-char fingerprints'. Like a FINGERPRINT: unique to each message, cannot recreate message from fingerprint!**

10. VIVA QUESTIONS & ANSWERS

Q: What is MD5?

A: Message Digest 5. A cryptographic hash function producing a 128-bit hash value.

Q: What is the output size of MD5?

A: 128 bits = 16 bytes = 32 hexadecimal characters.

Q: Is MD5 reversible?

A: No. It is a one-way function. You cannot get the original message from the hash.

Q: What is the avalanche effect?

A: A small change in input causes a drastically different hash output.

Q: Is MD5 secure?

A: No longer secure for cryptographic purposes. Vulnerable to collision attacks. SHA-256 is preferred.

Q: What is a collision in hashing?

A: When two different inputs produce the same hash output. MD5 has known collision vulnerabilities.

EXPERIMENT 8: IMPLEMENTATION OF SHA-1

1. CONCEPT (Simple Language)

- SHA-1 = Secure Hash Algorithm 1. Produces 160-bit (40 hex char) hash.
- More secure than MD5, but also deprecated now (SHA-256 is current standard).
- Works in 80 rounds on 512-bit blocks.
- 5 initial hash values (h0-h4), produces 20-byte output.

2. FULL C PROGRAM (Using OpenSSL)

```
#include<stdio.h>
#include<string.h>
#include<openssl/sha.h>

int main() {
char text[256];
unsigned char digest[SHA_DIGEST_LENGTH]; // 20 bytes
int i;
printf("Enter message: ");
fgets(text, sizeof(text), stdin);
text[strcspn(text, "\n")] = 0;
SHA1((unsigned char*)text, strlen(text), digest);
printf("SHA-1 Hash: ");
for(i = 0; i < SHA_DIGEST_LENGTH; i++)
printf("%02x", digest[i]);
printf("\n");
return 0;
}

// Compile: gcc sha1.c -o sha1 -lssl -lcrypto
// ---- Manual SHA-1 constants demo ----
#include<stdio.h>
#include<stdint.h>

// SHA-1 initial hash values
uint32_t h0 = 0x67452301;
uint32_t h1 = 0xEFCDAB89;
uint32_t h2 = 0x98BADCFE;
uint32_t h3 = 0x10325476;
uint32_t h4 = 0xC3D2E1F0;

// SHA-1 constants
// Round 0-19: K = 0x5A827999
// Round 20-39: K = 0x6ED9EBA1
// Round 40-59: K = 0x8F1BBCDC
// Round 60-79: K = 0xCA62C1D6

int main() {
printf("SHA-1 Initial Hash Values:\n");
printf("H0 = %08x\n", h0);
printf("H1 = %08x\n", h1);
```

```
printf("H2 = %08x\n", h2);
printf("H3 = %08x\n", h3);
printf("H4 = %08x\n", h4);
printf("Output: 5 values x 32bits = 160 bits = 40 hex chars\n");
return 0;
}
```

3. LINE-BY-LINE EXPLANATION

- SHA_DIGEST_LENGTH = 20 (bytes) = 160 bits
- SHA1() → OpenSSL function: input, length, output buffer
- 5 initial values (h0-h4) are SHA-1 magic constants (same as MD5's but different values)
- 80 rounds in 4 groups of 20, each using different constants K
- Final hash = h0||h1||h2||h3||h4 concatenated = 160 bits

4. SAMPLE INPUT & OUTPUT

- Input: "hello"
- SHA-1: aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d
- Output is always 40 hex characters = 160 bits

5. HOW TO IDENTIFY IN EXAM

- Keywords: SHA, SHA-1, 160-bit, 40 hex, secure hash, 80 rounds
- Difference from MD5: output is 40 hex chars (SHA-1) vs 32 hex chars (MD5)

6. COMMON MISTAKES

- Using wrong header: SHA-1 needs `<sha1.h>`, MD5 needs `<md5.h>`
- Confusing SHA_DIGEST_LENGTH (20) with MD5_DIGEST_LENGTH (16)
- Forgetting `-lssl -lcrypto` in compilation

7. 1-MINUTE REVISION SUMMARY

SHA-1: 160-bit hash = 20 bytes = 40 hex chars. 5 initial hash values (h0-h4). 80 rounds. Use OpenSSL: `SHA1(input, len, output)`. Compile with `-lssl -lcrypto`. More secure than MD5 but SHA-256 is now standard. One-way, avalanche effect applies.

8. MEMORY TRICK

■ **Memory Trick: 'SHA-1 has 1 extra step over MD5: 160 bits vs 128 bits'. MD5=32 hex, SHA1=40 hex, SHA256=64 hex. Count the zeros: MD5-128, SHA1-160, SHA256-256!**

10. VIVA QUESTIONS & ANSWERS

Q: What is SHA-1?

A: Secure Hash Algorithm 1. Produces a 160-bit hash from any input message.

Q: What is the output size of SHA-1?

A: 160 bits = 20 bytes = 40 hexadecimal characters.

Q: How many rounds does SHA-1 have?

A: 80 rounds, divided into 4 groups of 20 with different constants.

Q: Difference between MD5 and SHA-1?

A: MD5=128 bits output, SHA-1=160 bits output. SHA-1 is more secure than MD5.

Q: Is SHA-1 secure now?

A: No. Collision attacks exist. SHA-256 or SHA-3 are recommended now.

Q: What are the 5 initial hash values used in SHA-1?

A: h0=0x67452301, h1=0xEFCDAB89, h2=0x98BADCFE, h3=0x10325476, h4=0xC3D2E1F0.

EXPERIMENT 9: DIGITAL SIGNATURE STANDARD (DSS) – Simple C Program

1. CONCEPT (Simple Language)

- DSS = Digital Signature Standard. Uses RSA or DSA to sign messages.
- Signing: sender uses PRIVATE key to sign the hash of the message.
- Verification: receiver uses sender's PUBLIC key to verify the signature.
- Ensures: Authenticity (who signed), Integrity (not tampered), Non-repudiation (can't deny).
- Simple implementation: RSA-based signing using hash + modular exponentiation.

2. FULL C PROGRAM

```
#include<stdio.h>
#include<string.h>
#include<math.h>

long long power(long long base, long long exp, long long mod) {
    long long result = 1;
    base %= mod;
    while(exp > 0) {
        if(exp % 2 == 1) result = result * base % mod;
        exp /= 2;
        base = base * base % mod;
    }
    return result;
}

// Simple hash: sum of ASCII values mod n
long long simpleHash(char *msg, long long n) {
    long long hash = 0;
    int i;
    for(i = 0; msg[i]; i++)
        hash = (hash + (unsigned char)msg[i]) % n;
    return hash;
}

int main() {
    long long p, q, n, phi, e, d;
    long long hash, signature, verified;
    char msg[100];
    // Key generation (same as RSA)
    p = 61; q = 53;
    n = p * q; // n = 3233
    phi = (p-1)*(q-1); // phi = 3120
    e = 17; // public exponent
    d = 2753; // private exponent (17*2753 mod 3120 = 1)
    printf("Public Key (e,n): (%lld, %lld)\n", e, n);
    printf("Private Key (d,n): (%lld, %lld)\n", d, n);
    printf("Enter message to sign: ");
```



```
scanf("%s", msg);
// Sign
hash = simpleHash(msg, n);
printf("Message Hash: %lld\n", hash);
signature = power(hash, d, n); // Sign with private key
printf("Signature: %lld\n", signature);
// Verify
verified = power(signature, e, n); // Verify with public key
printf("Verified Hash: %lld\n", verified);
if(verified == hash)
printf("Signature VALID! Message is authentic.\n");
else
printf("Signature INVALID! Message may be tampered.\n");
return 0;
}
```

3. LINE-BY-LINE EXPLANATION

- $p=61, q=53 \rightarrow$ hardcoded primes for demo ($e=17, d=2753$ precomputed)
- `simpleHash()` $\rightarrow$ simplified hash: sum of ASCII values mod n
- Signing: $\text{signature} = \text{hash}^d \bmod n \rightarrow$ use PRIVATE key
- Verification: $\text{verified} = \text{signature}^e \bmod n \rightarrow$ use PUBLIC key
- $\text{if}(\text{verified} == \text{hash}) \rightarrow$ signature matches, message is authentic
- Power function uses fast modular exponentiation to avoid overflow

4. SAMPLE INPUT & OUTPUT

- Message: "HELLO"
- Hash = $(72+69+76+76+79) \bmod 3233 = 372$
- Signature = $372^{2753} \bmod 3233 =$ some large value
- Verified = $\text{signature}^{17} \bmod 3233 = 372$ (same as hash)
- Output: Signature VALID!

5. HOW TO IDENTIFY IN EXAM

- Keywords: digital signature, sign, verify, authenticate, private key sign, public key verify
- Logic: Hash the message $\rightarrow$ Sign hash with private key $\rightarrow$ Verify with public key

6. COMMON MISTAKES

- Mixing up: SIGN with private key, VERIFY with public key (opposite of encryption)
- Not hashing the message before signing
- Using wrong key pair: p, q, e, d values must be consistent

7. 1-MINUTE REVISION SUMMARY

DSS = Hash message + Sign with private key. To verify: decrypt signature with public key $\rightarrow$ compare with hash. Sign: $S = H^d \bmod n$. Verify: $H' = S^e \bmod n$. If $H == H' \rightarrow$ valid. Provides: Authentication, Integrity, Non-repudiation.

8. MEMORY TRICK

■ **Memory Trick:** 'Sign with PRIVATE (only YOU can sign), Verify with PUBLIC (anyone can verify)'. Think of a SEAL: only the king (private key) seals documents, but everyone (public key) can verify the royal seal!

9. MANUAL OUTPUT CALCULATION

- Hash = sum ASCII values mod n
- Sig = Hash^d mod n (use calculator or repeated squaring)
- Verify = Sig^e mod n (should equal Hash)

10. VIVA QUESTIONS & ANSWERS

Q: What is a Digital Signature?

A: A mathematical scheme for verifying authenticity and integrity of digital messages/documents.

Q: Which key is used for signing?

A: The sender's PRIVATE key is used to sign.

Q: Which key is used for verification?

A: The sender's PUBLIC key is used by anyone to verify the signature.

Q: What is non-repudiation?

A: The signer cannot later deny having signed, because only they have the private key.

Q: What are the 3 security properties of DSS?

A: Authentication (who signed), Integrity (not changed), Non-repudiation (cannot deny signing).

Q: What is the difference between encryption and digital signature?

A: Encryption: public key encrypts, private key decrypts. Signature: private key signs, public key verifies.

Q: Why hash the message before signing?

A: Hashing produces a fixed-size digest. Signing the small hash is faster and works for any message size.

QUICK REFERENCE CARD – ALL ALGORITHMS

Algo	Type	Key Formula	Output
Caesar	Substitution	$(ch - A + key) \% 26 + A$	Shifted text
Playfair	Substitution	5x5 matrix, 3 rules	Pair cipher
Hill	Substitution	$C = K * P \bmod 26$	Matrix result
Rail Fence	Transposition	Zigzag across N rails	Rearranged text
DES	Block Sym	Feistel 16 rounds	64-bit blocks
AES	Block Sym	$SB + SR + MC + ARK \times 10$	128-bit blocks
RSA	Asymmetric	$C = M^e \bmod n, M = C^d \bmod n$	Cipher number
D-H	Key Exchange	$g^{(ab)} \bmod p$	Shared secret
MD5	Hash	One-way function	128-bit/32 hex
SHA-1	Hash	One-way function	160-bit/40 hex
DSS	Signature	$S = H^d \bmod n, V = S^e \bmod n$	Valid/Invalid

KEY DIFFERENCES TO REMEMBER

Concept	Description
Symmetric vs Asymmetric	Sym: same key for enc/dec (DES, AES). Asym: different keys (RSA, DH)
Substitution vs Transposition	Sub: changes letters (Caesar, Playfair). Trans: rearranges (Rail Fence)
Hash vs Encryption	Hash: one-way, no key (MD5, SHA). Encrypt: reversible with key (AES, RSA)
Sign vs Encrypt	Sign: private key signs, public verifies. Encrypt: public encrypts, private decrypts
Block vs Stream	Block: fixed-size chunks (DES=64bit, AES=128bit). Stream: bit by bit

COMPILATION COMMANDS

Program	Compile Command
Caesar, Playfair, Hill, Rail Fence, DH, RSA, DSS	<code>gcc filename.c -o output</code>
RSA, DH (with math.h)	<code>gcc filename.c -o output -lm</code>
MD5 (OpenSSL)	<code>gcc filename.c -o output -lssl -lcrypto</code>
SHA-1 (OpenSSL)	<code>gcc filename.c -o output -lssl -lcrypto</code>
Run program	<code>./output</code>

Best of luck for your exam! You've got this! ■